

TelcoChurn MLOps

Interview Notes - 19 June 2026

HITS (what went right)

Root cause: ArgoCD self-heal was silently reverting all kubectl apply

Spent hours thinking 'kubectl apply -f' was broken. The real culprit? ArgoCD's spec.syncPolicy.automated.selfHeal: true kept restoring the git version (old manifests) within seconds of every apply. Lesson: always disable auto-sync before making changes, push to git, then re-enable.

Service discovery via Kubernetes API

Prometheus kubernetes_sd_configs needs RBAC permissions to list/watch pods. Docker-compose doesn't care (scrapes localhost). Created a dedicated ServiceAccount + ClusterRole with get/list/watch on pods, services, endpoints, nodes.

NodePort vs ClusterIP mental model

Docker-compose maps host:container directly. k3d uses a loadbalancer (serverlb) with iptables forwarding: localhost:3000 -> serverlb:30000 -> NodePort 30000 -> service:3000. If the service is ClusterIP, the NodePort target doesn't exist -> empty response.

Init containers for dependency ordering

Postgres might not be ready when Airflow starts. 'depends_on' in compose does this implicitly. In K8s, added wait-for-db init container running pg_isready. Also used a one-shot Job for airflow db init instead of running it in every pod.

ConfigMaps for file injection

Docker-compose bind-mounts files directly. K8s pods can't access host files. Created ConfigMaps for: DAGs (3 Python files), Prometheus config, Grafana datasources/dashboards, OTel collector config, Tempo config.

Policy-as-code with Kyverno

3 ClusterPolicies added: require prometheus.io/scrape annotation, require CPU/memory limits, disallow :latest tag. Runs in Audit mode for now.

Model retrain: sklearn vs xgboost dependency hell

Old pickle referenced xgboost even for sklearn RF model -> Docker image ballooned to 303MB (nvidia-nccl-cu12). Retrained with pure sklearn RandomForestClassifier -> image dropped to ~150MB. Also saved LabelEncoders because predict endpoint receives raw strings.

Separated serving vs training requirements

Root requirements.txt had 30+ deps (xgboost, mlflow, dvc, evidently). Split into serving/requirements.txt with only 12 runtime packages -> Docker build from 10min to 19s.

MISSES (what went wrong / what I'd do differently)

Reverted by ArgoCD for hours before diagnosing

Should have checked ArgoCD Application events first. 'kubectl describe application -n argocd telcochurn' would have shown the self-heal reverting changes immediately. Instead assumed 'kubectl apply' had a server-side bug.

Didn't check kubectl diff early enough

Would have shown that the live state matched git (not my local file) instantly. Use 'kubectl diff -f' before

'kubectl apply -f' as a sanity check.

Overloaded the cluster with too many services

25 pods on a single k3d node on a dev laptop -> 700% CPU usage. Should have marked non-critical services (airflow, kyverno reports-controller, argocd) as optional with replicas=0 by default.

Bitnami/kubectl init container didn't have RBAC

The wait-for-init container used bitnami/kubectl:latest but the default SA couldn't list jobs. Could have used curl + serviceaccount token against the K8s API directly instead.

Forgot Kyverno in initial architecture

Mentioned policy-as-code in the design but didn't install it until the user reminded me. Should have added it upfront with the initial cluster setup.

Grafana dashboard path mismatch

Provisioning config pointed to /var/lib/grafana/dashboards but ConfigMap was mounted at /etc/grafana/provisioning/dashboards. Dashboard JSONs were loaded but invisible.

Old ReplicaSets kept recreating pods

Deployments with new templates didn't scale down old ReplicaSets because new pods were stuck in Init. Had to manually delete old RS multiple times.

KEY TAKEAWAYS FOR INTERVIEWS

* ArgoCD self-heal is a feature, not a bug - but you must understand it before fighting it. The GitOps workflow is: disable sync -> change -> push -> re-enable sync.

* Kubernetes RBAC is not optional. Everything that talks to the API server needs explicit permissions (Prometheus SD, kubectl in init containers, ArgoCD itself).

* K3d's loadbalancer + NodePort model is different from Docker port mapping. The service must be type NodePort (or use port-forward) for external access.

* ConfigMaps are simple but don't support subdirectory mounting well. For multiple related config files, mount a separate ConfigMap per subdirectory or use a projected volume.

* Init containers solve the 'my database isn't ready' problem cleanly, but they block the pod lifecycle. Keep them simple (pg_isready, curl, sleep).

* Separating serving vs training dependencies is a low-effort, high-impact optimization (10min -> 19s Docker build).

* Always save preprocessing artifacts (LabelEncoders, scalers) alongside models. The serving endpoint receives raw data, not preprocessed features.

* Policy-as-code (Kyverno) should be one of the first things installed, not an afterthought - it catches issues before they reach production.